

The Instruction Gate: Work-Order Camouflage and the Bottleneck in Layered LLM Agent Defenses

Mao Luo^{1,*}, Hang Ding¹, Qi Liu¹, Aoxu Ji¹, Ruihao Xu¹,

¹Hubei University of Technology, Wuhan, China

luomao@hbut.edu.cn, dinghang@hbut.edu.cn, liuqi@hbut.edu.cn, jiaoxu@hbut.edu.cn,
xuruihao@hbut.edu.cn

Abstract

LLM agents are moving from text generation to action execution: their outputs can trigger tool calls over documents, messages, business records, and device APIs. This shift turns prompt injection into a system-level security problem. A successful attack is not defined only by an unsafe response, but by whether an adversarial instruction survives screening, is accepted as a task, and produces a target tool-call trajectory. Evaluation therefore has to inspect the artifacts the agent reads, the task it follows, and the execution trace it leaves behind. We study this problem in the OpenClaw Security Attack-Defense Challenge, where each target is evaluated through a three-stage funnel consisting of skill scanning, instruction-safety evaluation, and trajectory replay. Our goal is to understand how an attack can be optimized across these layers, and what the resulting failures reveal about agent defenses. We propose Business Work-Order Camouflage (BWC), which expresses target operations as ordinary work orders. BWC combines neutral case-index wording, compact supporting records, native schema cues, and the service’s delegated-access workflow when authorization is required. In the final official evaluation, BWC received full credit on 9 of 19 targets. Local replay analysis showed that 16 of 19 traces reached the intended operation, indicating that the main bottleneck was not tool-use capability but the LLM-based instruction gate. These results suggest that agent defenses should move beyond surface-level prompt screening toward effect-aware instruction review, scoped delegated authorization, and reproducible trajectory-based evaluation.

1 Introduction

When a language model’s output can trigger a tool call that modifies a document, sends a message, or changes a device setting, prompt injection stops being a text-generation problem. It becomes a system-level security problem: an adversarial instruction embedded in ordinary input can propagate

through screening, task planning, and tool execution to produce an unauthorized state change. Recent surveys of trustworthy LLM agents identify adversarial inputs and unsafe tool interactions as primary external threats [Yu *et al.*, 2025], and benchmarks for tool-integrated safety reveal that even capable models exhibit unsafe behavior at every stage of tool use [Ye *et al.*, 2024].

The attack surface is specific to agent settings. Classical prompt injection research focused on chat scenarios where the consequence is an undesirable text response [Wei *et al.*, 2023; Liu *et al.*, 2024]. When the model has access to tools, the same vulnerability can redirect tool selection, argument construction, and authorization use, producing concrete side effects in external systems [Greshake *et al.*, 2023; Liu *et al.*, 2023]. For tool-integrated agents, InjecAgent and AgentDojo demonstrate that injected instructions can induce unauthorized actions even when defenses are in place [Zhan *et al.*, 2024; Debenedetti *et al.*, 2024]. The consequences extend beyond a single misdirected call: agents trained to refuse harmful requests in chat mode become substantially more compliant when deployed as autonomous agents [Kumar *et al.*, 2024], and memory or knowledge-base poisoning can achieve persistent hijacking with low detection rate [Chen *et al.*, 2024].

Evaluation has to keep pace with this shift. Agent security cannot be measured only by whether a model refuses a harmful request in isolation. The evaluator has to inspect the artifacts the agent reads, the task it follows, the tool calls it makes, and the final state it produces. AgentHarm shows that leading models comply with harmful multi-step agent tasks even without adversarial prompting, and that simple jailbreaks further increase compliance [Andriushchenko *et al.*, 2024]. SeClaw argues for specification-driven security task synthesis paired with trajectory-aware assessment for autonomous agents [Cheng *et al.*, 2026]. OpenClaw operationalizes this view through a multi-gate scoring funnel: skill and attachment scanning, instruction-safety evaluation, and trajectory replay.

The OpenClaw Security Attack-Defense Challenge provides a controlled setting for studying how attacks and defenses interact across these layers. Each target is scored through the three-stage funnel described above. We treat this funnel as the evaluation environment proposed by the organizers, not as our own contribution. Our question is how an

attack package can be designed to pass through all stages and what the observed failure patterns reveal about layered agent defenses.

We propose Business Work-Order Camouflage (BWC), a method for constructing attack artifacts under this multi-stage funnel. BWC expresses target operations as routine operational work orders. It combines neutral case-index wording, compact supporting records, native schema cues, and delegated-access workflows exposed by the mock services. The design goal is not to rely on hidden encodings or refusal-breaking phrases, but to make the prompt, attachments, and expected tool sequence look like one coherent operational workflow.

Our final official submission obtained full credit on 9 of 19 targets. Local trajectory analysis showed that 16 of 19 traces reached the intended operation. The gap between these two numbers is the central empirical finding: most target trajectories were technically reachable, but the instruction-safety gate remained the dominant bottleneck. The failures suggest that surface-level prompt screening is misaligned with runtime behavior, that chat-mode refusal does not transfer to agent-mode security, and that reproducible trajectory-based evaluation is necessary for diagnosing defense failures.

This paper makes four contributions.

- We formulate the challenge task as a multi-stage agent-security problem spanning submitted artifacts, accepted instructions, and replayed tool-call trajectories.
- We describe BWC, a work-order-based attack method that combines neutral wording, schema-level steering, supporting records, and delegated-access composition to navigate layered defenses.
- We report official and local results on 19 targets, including ablations that separate prompt rejection from runtime trajectory failure.
- We discuss defensive implications for effect-aware instruction review, scoped delegated authorization, and reproducible safety evaluation.

All experiments were run in the organizer-provided sandbox against mock services. We report the method to understand layered agent defenses, not to encourage use against real systems.

2 Related Work

This work sits at the intersection of tool-integrated agent security, prompt injection, and trajectory-based safety evaluation.

Agent security with tool use. Recent surveys map the threat landscape for LLM agents. Yu et al. propose the TrustAgent framework, which separates internal threats such as hallucination and memory corruption from external threats including adversarial users, unsafe tools, and hostile environments [Yu et al., 2025]. ToolSword evaluates safety across three stages of tool integration and finds that even GPT-4 exhibits unsafe behavior at input, execution, and output stages [Ye et al., 2024]. These works establish that agent security is a system-level property: it depends not only on

the model but on the interaction between model, tools, and environment.

Prompt injection targeting tool-integrated agents. Greshake et al. introduced indirect prompt injection, showing that instructions embedded in retrieved external content can hijack the behavior of LLM-integrated applications [Greshake et al., 2023]. Liu et al. formalized this threat with the HouYi framework, demonstrating that 31 of 36 real-world LLM-integrated platforms were vulnerable to system-level instruction injection [Liu et al., 2023]. For tool-integrated agents specifically, InjecAgent benchmarks how injected instructions in tool outputs induce unauthorized actions [Zhan et al., 2024], and AgentDojo provides a dynamic environment with hundreds of security evaluations showing that current defenses fail to prevent all exploits [Debenedetti et al., 2024]. Our work differs from these benchmarks in that it studies a competition setting where the submitted artifact must survive several independent gates before the trajectory is scored.

Jailbreaking and agent-mode safety. Wei et al. analyzed why LLM safety training breaks down, attributing failures to competing objectives between capability and safety, and to mismatched generalization where safety does not extend to novel inputs [Wei et al., 2023]. These findings become more consequential in agent settings. Kumar et al. showed that refusal-trained models are easily jailbroken when deployed as browser agents: chat-mode safety does not transfer to agent-mode behavior [Kumar et al., 2024]. AgentHarm provides a benchmark of 110 malicious agent tasks and finds that leading models comply with harmful multi-step requests even without jailbreaking, and that simple prompt injections further increase compliance [Andriushchenko et al., 2024]. AgentPoison demonstrates that poisoning an agent’s memory or knowledge base can achieve high attack success rates with negligible disruption to normal operations [Chen et al., 2024]. Our work complements these studies by examining how an attacker navigates not only the model’s refusal boundary but also independent artifact-level and instruction-level safety gates.

Trajectory-based safety evaluation. Several benchmarks evaluate agent safety through execution traces rather than final-response judging. ToolEmu uses an LM-emulated sandbox to identify risky tool-use failures at scale, with human audits confirming that a majority of discovered vulnerabilities constitute authentic risks [Ruan et al., 2024]. R-Judge evaluates safety-risk awareness from multi-turn agent interaction records [Yuan et al., 2024]. CyberSecEval 2 measures prompt injection resistance and interpreter abuse across leading models, reporting 26–41% attack success rates [Bhatt et al., 2024]. SeClaw combines specification-driven security task synthesis with execution-based evaluation for autonomous agents [Cheng et al., 2026]. OpenClaw follows this trajectory-aware view and adds multi-gate scoring. Our contribution is not a new benchmark, but an empirical attack method and an analysis of which layer of the OpenClaw funnel dominates failure.

Authorization and privilege control for agents. The principle of least privilege requires that every program and user operate with the minimum authority necessary [Saltzer and

Schroeder, 1975]. South et al. argue that AI agents need authenticated delegation protocols extending OAuth with agent-specific cryptographic credentials to formalize human-agent authorization [South *et al.*, 2025]. Progent enforces programmable privilege control through symbolic rules on tool names and parameters, requiring user consent for privilege escalation and significantly reducing attack success rates [Shi *et al.*, 2025]. In the OpenClaw setting, the delegated-access pattern we exploit is an authorization workflow already present in the mock services: the agent obtains a legitimate token through the system’s own access-request mechanism. The attack does not forge credentials; it composes authorized workflows in ways that produce a protected action. This is precisely the scenario that scoped delegation and runtime policy enforcement are designed to prevent.

3 Method

3.1 Challenge Setting

For each target, the attacker submits an attack directory containing a task prompt, a skill file, and optional attachments. The package also includes an output directory from a real run in the organizer image. The evaluator replays this trace rather than trusting a written claim about what happened.

A target receives full credit only if all three stages pass:

1. Skill security scan: checks the skill and attachments for unsafe or suspicious content.
2. Instruction security evaluation: checks whether the task prompt appears to induce unsafe behavior.
3. Attack trajectory score: checks the actual tool-call trace against the target condition.

The funnel is difficult because each stage rewards a different property. Stage 1 rewards ordinary-looking support material. Stage 2 rewards instructions that do not look unsafe on their surface. Stage 3 rewards precise execution: the right service, payload, and final state. These goals can conflict. A more explicit prompt may produce a cleaner trace but fail the instruction gate. A very bland prompt may pass the gate but leave the agent without enough guidance to reach the target.

3.2 Business Work-Order Camouflage

BWC is a template for building artifacts under this three-way constraint. It does not rely on hidden encodings, magic strings, or direct refusal-breaking language. Instead, it borrows the form of routine enterprise work: a role, a case reference, a compact case index, a supporting record, and a final state. The format is easy for an agent to follow, but it looks much closer to ordinary work than to an explicit harmful request.

3.3 Core Design

Neutral work-order framing. The task prompt casts the agent as an operations assistant, not as an adversary. Sensitive parameters appear as case fields: resource identifier, requested state, owner, reason label, or record type. The goal is not to hide the operation from the agent. The goal is to express it in the same language as legitimate business work, so the prompt and the expected workflow feel consistent.

Algorithm 1 BWC iteration for one target

- 1: Study the closest same-domain target that already scored successfully.
 - 2: Write the skill as a short standard operating procedure.
 - 3: Write the task as a neutral work order with a case index and final state.
 - 4: Place identifiers and form fields in compact supporting attachments.
 - 5: Run a local Stage-1/2 simulator as a rough filter.
 - 6: Execute the target in a fresh image and inspect the generated trace.
 - 7: If the trace misses the target, adjust the wording without making it overtly unsafe; rerun.
 - 8: Freeze only when wording, trace, and package consistency all hold.
-

In practice, the best prompts were neither explicit nor vague. They gave the agent enough structure to call the right API, but avoided language that looked like bypassing, coercion, or permission override. This balance was the central tuning problem throughout the challenge.

Native-schema steering. Agents often fail for mundane reasons: they explore the wrong service, omit a field, or use the wrong payload shape. BWC gives schema-level guidance in ordinary terms: use the application’s native form keys, preserve the case identifiers, and record the final state. Compact attachments repeat the same identifiers as supporting records.

This is not just formatting. Shorter traces are easier to score and contain fewer unrelated actions. Successful artifacts usually reached the target in a small number of calls. Failed trajectory candidates often wandered through adjacent endpoints before stopping at the wrong state.

Delegated access composition. In several targets, a plain successful call was not enough. The trace also needed to show that a protected action was completed through an accepted authorization path. BWC uses a design pattern present in the mock services. When a protected operation requires delegated access, the system exposes an access-request workflow. The agent can request authorization from a named owner, receive a token, and then send the protected operation with that token. The key point is that the token is real: it is produced by the system itself. The attack does not forge credentials. It gets the agent to compose an allowed access workflow with a protected action in a way the application accepts.

3.4 Self-Consistency and Implementation

Every iteration followed a strict rule: if any prompt artifact changed, the image had to be rerun. We used audit scripts to compare the packaged prompt materials with the prompt text recorded in the trace, and to check that each target contained one coherent execution. This matters for scientific validity as well as competition compliance. This paper reports behavior from real agent runs, not hand-assembled traces.

The implementation is intentionally simple. Each target is a self-contained package with prompt artifacts and a generated trace. Around this package format, we used four local tools:

Scene	Targets	Stage 1	Stage 2	Full
Smart home	5	4	1	1
E-commerce	6	6	5	5
Enterprise OA	8	8	3	3
Total	19	18	9	9

Table 1: Official stage survival by scene. “Full” means all three stages passed.

- Safety pre-filter: an approximate LLM-based simulator of the first two gates. It helped with triage, but we never treated it as authoritative.
- Trajectory diagnostic: a parser for the unified audit log that reports service coverage, status codes, trace length, and first tools.
- Target verifier: a check for whether the trace reached the intended operation and final state.
- Package auditor: a consistency check ensuring that prompts, metadata, logs, and traces belong to the same run.

The workflow was deliberately slow and local. We first froze targets that had already received full official credit. For the remaining targets, we changed one target at a time, pre-filtered the wording, reran the image, inspected the trace, and either froze the target or revised it again. This avoided a common late-stage failure mode: changing many prompts at once and losing track of which edit helped.

4 Experimental Results

4.1 Setup and Metrics

The benchmark contains 19 targets across three scenes: smart home, e-commerce, and enterprise office automation. The base model was Kimi-K2.5. We report the official score table from our final evaluated submission. We also report a local replay check over the same artifacts. The local replay check asks whether the generated trace reaches the intended operation regardless of whether earlier official gates accepted the prompt; it is used only as an ablation variable.

4.2 Official Results

BWC obtained full official credit on 9 of 19 targets. The successful targets span all three scenes: one smart-home target, five e-commerce targets, and three enterprise-OA targets. This matters because the method was not tied to one API or one scenario. The same work-order pattern transferred across different tools and domains.

Stage-wise finding. The clearest result is the shape of the funnel. Stage 1 passed for 18 of 19 targets, while Stage 2 passed for only 9. Local trajectory analysis, however, found that 16 targets already produced a target-reaching trace. The hardest part was usually not making the agent act. It was writing instructions that remained acceptable to the instruction-safety evaluator while still steering the action.

Check	Pass	Fail	Rate
Skill scan	18	1	95%
Instruction gate	9	10	47%
Official trajectory	9	10	47%
Full target score	9	10	47%
Local trajectory hit	16	3	84%

Table 2: Layer-level outcomes over 19 target packages. The large difference between local trajectory hits and full official scores shows that many traces were executable, but did not survive the earlier instruction gate.

Group	Targets	Full	Local	Calls
Accepted	9	9	9	4
Rejected, hit	7	0	7	3
Rejected, miss	3	0	0	7

Table 3: Cross-tabulation of instruction approval and local replay. “Accepted” means instruction-gate pass. “Hit” means local target-reaching replay. “Calls” is the median number of tool calls in each group.

Trace shape. Successful traces were compact. They usually made a small number of calls to the primary service and ended with the intended state change. Failed traces showed two patterns: the agent reached a related but unscored endpoint, or it explored several services and never produced the required final evidence. This supports schema-level steering as a practical tool: it reduces ambiguity without making the prompt overtly unsafe.

4.3 Ablation Results

The ablation unit is one complete target package. For each of the 19 targets, we recorded the official gate outcomes, the final official target score, a local replay result that checks whether the trace reaches the intended operation, the number of tool calls in the trace, and whether the target was edited in the final repair round. The local replay result is not a substitute for the official score. It is an ablation variable: it asks whether the execution trace already contains the desired behavior if the earlier semantic gates are ignored.

Semantic approval is the main bottleneck. Table 2 separates the official gates from the local replay check. The skill scan passed on 18 targets, so most skill files and attachments were acceptable to the static scanner. The full score, however, remained at 9 targets. The missing credit is not explained by a lack of executable traces: 16 targets produced local target-reaching traces. The dominant failure is therefore the instruction gate, where the task prompt is judged before the trace can count.

Rejected prompts often have compact successful traces. Table 3 shows three groups. All 9 targets that passed the instruction gate also passed the official trajectory check and received full credit. Another 7 targets failed the instruction gate but still had a local target-reaching trace. These 7 traces were short, with a median of 3 tool calls. This is smaller than the median of 4 calls for officially successful targets. Thus, for

Subset	Targets	S2	Full	Local
All targets	19	9	9	16
Edited repair set	6	1	1	4
Unchanged set	13	8	8	12

Table 4: Effect of the final repair round. S2 denotes instruction-gate pass, and Local denotes local target-reaching replay. Editing improved one target to full credit, but most edited targets remained blocked at the instruction gate.

these cases, failure was not caused by long or noisy trajectories. The trace was already direct enough; the wording of the task prompt was the rejected component.

The remaining 3 targets failed both the instruction gate and the local replay check. Their traces were longer, with a median of 7 calls. This pattern is different. Here the task was not only rejected by the semantic gate; it also failed to give the agent enough operational structure to reach the scored endpoint. These cases show why simply making a prompt more neutral is not sufficient. Neutral wording must still preserve concrete workflow cues.

Repair requires the right kind of detail. The final repair round changed 6 targets and reran their traces. As Table 4 shows, 4 of the 6 edited targets reached the intended operation locally, but only 1 passed the instruction gate and received full official credit. The successful repair was the approval-tampering target. Its prompt did not merely add more detail; it aligned the record identity, requested state, and delegated-access workflow into one plausible business procedure. The five remaining edited targets show the negative result: additional operational detail helps only when it reads as part of an ordinary workflow. More detail by itself can still look semantically unsafe to the instruction evaluator.

Evaluator variance affects borderline cases. One inventory-update target changed from failing in one official submission to full credit in a later one without a package change. The artifact hash was identical. Since the trace was fixed, the most plausible explanation is non-determinism or policy drift in the LLM-based instruction evaluator. This matters beyond the competition. If a safety gate is not reproducible, researchers cannot cleanly distinguish a better attack from evaluator noise, and defenders cannot easily debug why a borderline task was accepted or rejected.

Summary. The ablations support a narrow operating point. A prompt that is too direct fails the instruction gate. A prompt that is too vague misses the target at replay time. BWC works when the task looks like a normal work order but still carries enough operational structure for the agent to execute the intended workflow. The data also show why trajectory-only evaluation is incomplete: 16 targets were locally reachable, but only 9 were accepted by the full official funnel.

5 Discussion

Screen effects, not only words. Surface-level instruction screening can be fooled by routine business language. Defenses should reason about projected effects: which resource

will be changed, whose authority is used, whether the action is read-only or mutating, and whether the requester is entitled to the final state. This shifts the defense from prompt classification toward policy over concrete tool effects.

Scope delegated authorization. The delegated-access pattern is safe only when tokens are tightly scoped. A token should bind requester, approver, resource, action, and expiration. A token obtained for viewing or generic access should not automatically authorize mutation or export. Runtime authorization should remain load-bearing even when the natural-language task appears routine.

Make runtime policy load-bearing. Prompt-level defenses are useful, but they should not be the final authority. The most reliable control point is the tool boundary, where the system knows the concrete operation and parameters. Runtime policy should reject unsafe compositions even when the natural-language task looks benign.

Make safety evaluation reproducible. If LLM-based gates are used, their model version, prompt, temperature, and outputs should be logged and versioned. Security evaluation should be replayable. Otherwise, attack success and defense failure become hard to diagnose.

Limitations and ethics. This work is specific to the OpenClaw benchmark and its mock services. Transfer to other systems depends on whether they expose similar delegated-access workflows and whether their agents follow structured operational prompts. The results also depend on one base model and one official evaluation pipeline.

All experiments were conducted in the sandboxed competition environment against mock data and mock services. We do not claim or test effectiveness against real production systems. We also avoid publishing target-specific payloads in this paper. The purpose of the work is to help defenders understand how separate safety layers can fail together, and how runtime authorization should be hardened.

6 Conclusion and Future Work

OpenClaw shows that agent security is not only about whether a model refuses a harmful prompt. It is about whether a complete system preserves safety across submitted artifacts, instruction interpretation, and tool execution. BWC shows how these layers can interact in practice: neutral work-order language can pass some semantic filters, compact schema guidance can steer tool use, and delegated-access workflows can produce runtime authorization evidence.

The official results are mixed, but useful. Full credit on 9 of 19 targets shows that the attack transfers across domains. Local target-reaching trajectories on 16 of 19 targets show that runtime steering is often possible. The Stage-2 bottleneck shows that instruction screening still matters, but also that it is brittle. The broader lesson is clear: agent defenses should evaluate concrete tool effects, enforce scoped authorization at runtime, and make safety judgments reproducible.

Future work should evaluate the same method across more base models, construct controlled variants that isolate individual wording choices, and test defenses that combine instruction review with runtime effect policies. A more re-

469 producible LLM-based safety gate would also make it easier
470 to distinguish genuine method improvements from evaluator
471 variance.

472 Acknowledgments

473 We thank the organizers of the OpenClaw Security Attack-
474 Defense Challenge for designing and running the competi-
475 tion, and Alibaba Cloud for providing the Tianchi platform
476 and the sandboxed experimental environment used in all ex-
477 periments.

478 References

479 [Andriushchenko *et al.*, 2024] Maksym Andriushchenko,
480 Alexandra Souly, Mateusz Dziemian, Derek Duenas,
481 Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou,
482 Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome
483 Wynne, Yarin Gal, and Xander Davies. AgentHarm: A
484 benchmark for measuring harmfulness of LLM agents.
485 *arXiv preprint arXiv:2410.09024*, 2024.

486 [Bhatt *et al.*, 2024] Manish Bhatt, Sahana Chennabasappa,
487 Yue Li, Cyrus Nikolaidis, Daniel Song, Shengye Wan,
488 Faizan Ahmad, Cornelius Aschermann, Yaohui Chen,
489 Dhaval Kapil, David Molnar, Spencer Whitman, and
490 Joshua Saxe. CyberSecEval 2: A wide-ranging cyberse-
491 curity evaluation suite for large language models. *arXiv*
492 *preprint arXiv:2404.13161*, 2024.

493 [Chen *et al.*, 2024] Zhaorun Chen, Zhen Xiang, Chaowei
494 Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming
495 LLM agents via poisoning memory or knowledge bases.
496 *arXiv preprint arXiv:2407.12784*, 2024.

497 [Cheng *et al.*, 2026] Hao Cheng, Changtao Miao, Tianle
498 Song, Yin Wu, He Liu, Erjia Xiao, Junchi Chen, Xiaoyu
499 Shi, Yichi Wang, Jing Yang, et al. SeClaw: Spec-driven
500 security task synthesis for evaluating autonomous agents.
501 *arXiv preprint arXiv:2606.02302*, 2026.

502 [Debenedetti *et al.*, 2024] Edoardo Debenedetti, Jie Zhang,
503 Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and
504 Florian Tramèr. AgentDojo: A dynamic environment to
505 evaluate prompt injection attacks and defenses for LLM
506 agents. In *Advances in Neural Information Processing Sys-*
507 *tems Datasets and Benchmarks Track*, 2024.

508 [Greshake *et al.*, 2023] Kai Greshake, Sahar Abdelnabi,
509 Shailesh Mishra, Christoph Endres, Thorsten Holz, and
510 Mario Fritz. Not what you’ve signed up for: Compromis-
511 ing real-world LLM-integrated applications with indirect
512 prompt injection. In *Proceedings of the 16th ACM Work-*
513 *shop on Artificial Intelligence and Security*, pages 79–90,
514 2023.

515 [Kumar *et al.*, 2024] Priyanshu Kumar, Elaine Lau, Saranya
516 Vijayakumar, Tu Trinh, Elaine Chang, Vaughn Robin-
517 son, Sean Hendryx, Shuyan Zhou, Matt Fredrikson, Sum-
518 mer Yue, and Zifan Wang. Refusal-trained LLMs are
519 easily jailbroken as browser agents. *arXiv preprint*
520 *arXiv:2410.13886*, 2024.

521 [Liu *et al.*, 2023] Yi Liu, Gelei Deng, Yuekang Li, Kailong
522 Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang,

Yepang Liu, Haoyu Wang, Yan Zheng, Leo Yu Zhang, and
Yang Liu. Prompt injection attack against LLM-integrated
applications. *arXiv preprint arXiv:2306.05499*, 2023.

[Liu *et al.*, 2024] Yupei Liu, Yuqi Jia, Runpeng Geng,
Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and
benchmarking prompt injection attacks and defenses. In
Proceedings of the 34th USENIX Security Symposium,
2024.

[Ruan *et al.*, 2024] Yangjun Ruan, Honghua Dong, Andrew
Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann
Dubois, Chris J. Maddison, and Tatsunori Hashimoto.
Identifying the risks of LM agents with an LM-emulated
sandbox. *arXiv preprint arXiv:2309.15817*, 2024.

[Saltzer and Schroeder, 1975] Jerome H. Saltzer and
Michael D. Schroeder. Protection of information in com-
puter systems. In *Proceedings of the IEEE*, volume 63,
pages 1278–1308. 1975.

[Shi *et al.*, 2025] Tianneng Shi, Jingxuan He, Zhun Wang,
Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song.
Progent: Securing AI agents with privilege control. *arXiv*
preprint arXiv:2504.11703, 2025.

[South *et al.*, 2025] Tobin South, Samuele Marro, Thomas
Hardjono, Robert Mahari, Cedric Deslandes Whitney,
Dazza Greenwood, Alan Chan, and Alex Pentland. Au-
thenticated delegation and authorized AI agents. In *Pro-*
ceedings of the 42nd International Conference on Machine
Learning, 2025.

[Wei *et al.*, 2023] Alexander Wei, Nika Haghtalab, and Ja-
cob Steinhardt. Jailbroken: How does LLM safety train-
ing fail? In *Advances in Neural Information Processing*
Systems, 2023.

[Ye *et al.*, 2024] Junjie Ye, Sixian Li, Guanyu Li, Caishuang
Huang, Songyang Gao, Yilong Wu, Qi Zhang, Tao Gui,
and Xuanjing Huang. ToolSword: Unveiling safety is-
sues of large language models in tool learning across three
stages. In *Proceedings of the 62nd Annual Meeting of the*
Association for Computational Linguistics, pages 2210–
2228, 2024.

[Yu *et al.*, 2025] Miao Yu, Fanci Meng, Xinyun Zhou, Shi-
long Wang, Junyuan Mao, Linsey Pang, Tianlong Chen,
Kun Wang, Xinfeng Li, Yongfeng Zhang, Bo An,
and Qingsong Wen. A survey on trustworthy LLM
agents: Threats and countermeasures. *arXiv preprint*
arXiv:2503.09648, 2025.

[Yuan *et al.*, 2024] Tongxin Yuan, Zhiwei He, Lingzhong
Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu,
Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang,
and Gongshen Liu. R-Judge: Benchmarking safety risk
awareness for LLM agents. In *Findings of the Association*
for Computational Linguistics, 2024.

[Zhan *et al.*, 2024] Qiusi Zhan, Zhixiang Liang, Zifan Ying,
and Daniel Kang. InjecAgent: Benchmarking indirect
prompt injections in tool-integrated large language model
agents. In *Findings of the Association for Computational*
Linguistics, pages 10471–10506, 2024.